Retrieval Engine Competition Project

Notebook Report

Group: SeaFour

Maksym DOLHOV
Mehdi AGHAEI
Nguyen Ho Bao KHANH

Report objective. This version updates the written report to match the current saved notebook exactly, including its present configuration choices and its current leakage-safe evaluation protocol. The active configuration uses embedding first-stage retrieval, a cross-encoder second-stage reranker, and a TF-IDF + LinearSVC category classifier fit only on the split-train queries, whose prediction is converted into a soft category bonus during reranking.

Primary source files. `kaggle-submissione.ipynb`, `solutions_SeaFour.csv`, and this LaTeX report.

Notebook file	`kaggle-submissione.ipynb`
Submission file	`solutions_SeaFour.csv`
Current default model	`embedding`
Submission size target	Top 7,500 first-stage candidates; rerank top 150 with the same train-fitted artifacts for validation, hold-out test, and production
Date	March 28, 2026

1 Notebook-Aligned System Overview

The logic is organized into several stages: strict data loading, shared text normalization, a stratified 60/30/10 split over the labeled queries, sequential model experimentation (TF-IDF to BM25+ to embeddings), aggressive caching for performance, category-classifier fitting on the split-train queries, cross-encoder training on the split-train queries, validation-based model selection, one hold-out test evaluation, and final submission generation.

The current system relies on a dense first-stage embedding retriever followed by a cross-encoder reranker, with category information injected as a soft bonus.

The present notebook snapshot is configured as follows:

- `EVALUATION_TOP_KS = (7500,)`
- `TRAIN_FRACTION = 0.60`, `VALIDATION_FRACTION = 0.30`, `TEST_FRACTION = 0.10`
- `CROSS_ENCODER_TRAIN_QUERY_LIMIT = 327`
- `CROSS_ENCODER_RERANK_TOP_M = 150` for validation, hold-out test, and production reranking
- category bonus = 2.0 whenever `ENABLE_CATEGORY_BOOST = True`
- the same train-fitted classifier vectorizer and cross-encoder are reused unchanged on validation, hold-out test, and production queries

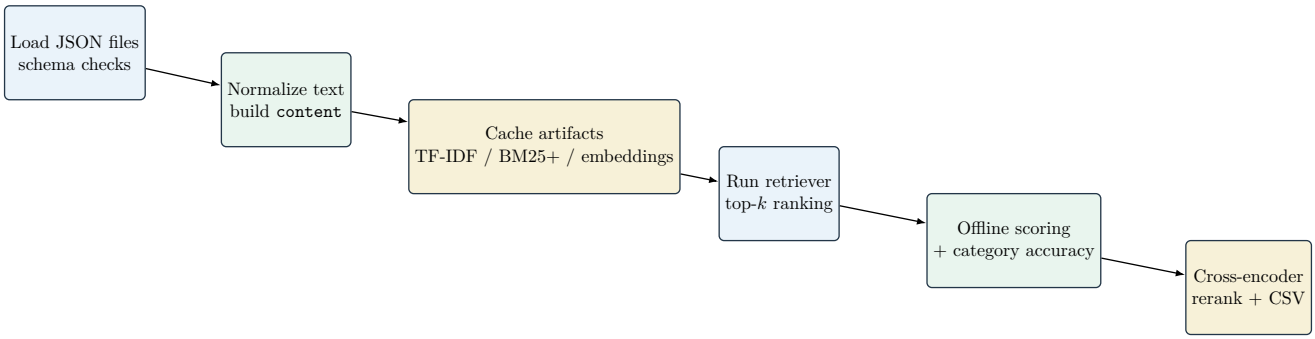

Figure 1: High-level end-to-end workflow in the saved notebook.

Component	Implemented	Active in saved run	Role in the pipeline
TF-IDF retrieval	Yes	No	Sparse lexical baseline using cosine similarity. Fast, but struggled with semantic mismatch.
BM25+ retrieval	Yes	No	Stronger lexical baseline with token-level scoring and document-length normalization.
Embedding retrieval	Yes	Yes	Dense semantic first-stage retrieval with Sentence-Transformers. Yielded the best metrics.
Cross-encoder reranker	Yes	Yes	Re-ranks the first-stage candidates using joint query-document scoring. The current notebook uses top-$m = 150$ during validation, hold-out test, and production submission generation.
Category classifier	Yes	Yes	Fits a TF-IDF + LinearSVC classifier on the split-train query set, reuses the same fitted transform on validation, hold-out test, and production queries, and provides a soft category bonus of 2.0 during reranking when enabled.

Table 1: What the notebook supports versus what the saved execution actually uses.

2 Dataset and Preprocessing

The notebook loads 216,041 documents, 327 training queries, 141 test queries, and one training qrels file. The document collection is distributed across five categories, with significant class imbalance.

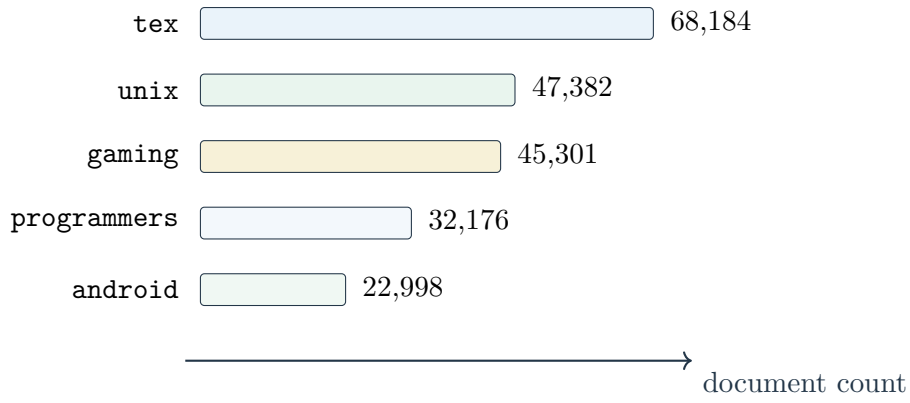

Figure 2: Document distribution by category.

The notebook now splits the 327 labeled queries into 196 split-train queries, 98 validation queries, and 33 hold-out test queries. Because 327 is not divisible exactly into 60/30/10, these are the closest integer counts, corresponding to 59.94%, 29.97%, and 10.09%.

Text normalization is shared across all components: separators `-`, `_`, and `/` are replaced by spaces, whitespace is collapsed, and text is lowercased. Retrieval documents are built from `title + text + tags`, while retrieval queries use only `title + text`. The classifier uses `title + text + tags` on the query side, but its fitted TF-IDF transform is learned only on the split-train query set and then reused with `transform(...)` on validation, hold-out test, and production queries. This separation keeps the retrieval input closer to what the system will actually receive at test time while still allowing the classifier to exploit the extra categorical signal without leaking fitted preprocessing across splits. The

saved notebook prints average normalized lengths of 900.9 characters for documents, 48.4 for training queries, and 48.9 for test queries. This length disparity informed our ultimate shift away from purely lexical methods.

3 Experimental Progression and Pipeline Optimization

Our methodology evolved through a structured series of experiments, beginning with simple baselines and moving toward a complex, optimized semantic architecture.

3.1 Phase 1 & 2: Lexical Baselines (TF-IDF and BM25+)

We initially implemented a **TF-IDF** retriever (using unigrams/bigrams with `min_df=2`). This served as a fast, lightweight baseline to validate our data loading and evaluation pipeline. While transparent, TF-IDF struggled significantly with semantic mismatch – when queries paraphrased document concepts without sharing exact vocabulary, recall dropped.

To improve this, we upgraded to **BM25+**. By tokenizing the text and applying `k1=1.5` and `b=0.75`, BM25+ provided much stronger lexical ranking, correcting for the high variance in document lengths. In the current notebook configuration, lexical components also use English stopword filtering. However, BM25+ still fundamentally relied on exact token overlap, meaning synonymous phrasing remained a weakness.

3.2 Phase 3: The Semantic Shift (Dense Embeddings)

To overcome the vocabulary mismatch problem, we transitioned to dense semantic retrieval using the Sentence-Transformer model `all-MiniLM-L6-v2`. Documents and queries were encoded into 384-dimensional L2-normalized vectors, allowing us to retrieve documents based on contextual meaning rather than exact keywords. This dramatically improved recall.

3.3 Phase 4: Overcoming the 1.5-Hour Bottleneck

The shift to embeddings introduced a severe computational roadblock: initially, encoding the 216,041 documents and calculating scores took almost **1.5 hours to run**. This latency crippled our ability to run rapid iterations.

To solve this, we implemented aggressive caching and chunking mechanisms:

- **Disk Caching:** We engineered a caching system that fingerprints the dataframes and saves the expensive document embeddings as `.npy` arrays. Repeated notebook runs now load the matrix in seconds instead of re-encoding.
- **Chunked Query Scoring:** Instead of creating massive, memory-crashing score blocks, we scored queries in chunks of 32 (`EMBEDDING_QUERY_CHUNK_SIZE = 32`).
- **Partial Sorting:** We replaced full array sorts with `np.argpartition` to extract only the top-K candidates efficiently.

3.4 Phase 5: Current Two-Stage Reranking Configuration

With the dense pipeline running smoothly, the notebook fixes the first-stage retrieval depth to `top_k = 7,500` for both offline evaluation and submission generation. The current notebook snapshot evaluates only the `embedding` retriever (`EVALUATION_MODELS = ("embedding",)`) and stores one offline configuration table for that model.

To refine the first-stage ranking, the notebook adds two auxiliary components:

- a TF-IDF + LinearSVC category classifier trained on the split-train query labels;
- a cross-encoder reranker trained on the split-train queries only.

More specifically:

- the notebook builds a stratified split over labeled queries before any fitted query-side preprocessing is learned;
- the classifier training frame is `split_train_queries_classifier_df`;
- classifier content is built from `title + text + tags`;
- query-side classifier frames also use `title + text + tags`;
- the retriever itself continues to use only `title + text`;
- `CROSS_ENCODER_TRAIN_QUERY_LIMIT = 327`, but the actual available split-train scope is 196 queries;
- `CROSS_ENCODER_HARD_NEGATIVE_TOP_K = 200`;
- up to four positives per query and four negatives per positive are used for cross-encoder training;
- validation, hold-out test, and production reranking all use `CROSS_ENCODER_RERANK_TOP_M = 150`;
- the same fitted classifier vectorizer and trained cross-encoder are reused on production queries without refitting.

The category signal is no longer applied as a strict hard filter inside the active offline loop. Instead, the notebook computes

$$\texttt{current\_bonus} = \begin{cases} 2.0 & \text{if } \texttt{ENABLE\_CATEGORY\_BOOST = True} \\ 0.0 & \text{otherwise} \end{cases}$$

and adds that scalar bonus to cross-encoder scores for documents whose category matches the predicted query category.

4 Evaluation Protocol and Results

The current saved notebook now follows a leakage-safe protocol built around a stratified 60/30/10 split over the 327 labeled queries. More precisely, it performs the following sequence:

1. build a stratified split with 196 split-train queries, 98 validation queries, and 33 hold-out test queries,
2. fit the TF-IDF + LinearSVC category classifier only on `split_train_queries_classifier_df`,
3. compute category predictions for validation, hold-out test, and production query frames with the same fitted transform,
4. train or load the cross-encoder on `split_train_queries_df` and `train_ground_truth`,
5. run retrieval on the validation queries at `top_k = 7,500`,
6. rerank the top 150 documents per validation query with the cross-encoder and the soft category bonus,
7. choose the best validation configuration, then run it once on the hold-out test split,
8. generate production predictions for the 141 Kaggle test queries using the same fitted classifier vectorizer and the same trained cross-encoder.

So the present notebook's reported offline comparison is a **validation-first, hold-out-tested estimate** rather than the older training-query score used in previous versions. This distinction matters because the fitted query-side preprocessing and the cross-encoder no longer see the validation or hold-out test

queries during fitting. The report now follows the cleaned notebook rather than the older notebook snapshot.

The combined score is calculated as:

$$\text{Score} = \frac{1}{4}\left(\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy}\right)$$

Setting	Current notebook value
`EVALUATION_MODELS`	`("embedding",)`
`EVALUATION_TOP_KS`	`(7500,)`
Query split	196 split-train / 98 validation / 33 hold-out test queries
Category-classifier fit scope	`split_train_queries_classifier_df` only
Cross-encoder train scope	`split_train_queries_df` with `train_ground_truth` (196 queries)
Validation rerank depth	`CROSS_ENCODER_RERANK_TOP_M = 150`
Hold-out test rerank depth	`CROSS_ENCODER_RERANK_TOP_M = 150`
Production rerank depth	`CROSS_ENCODER_RERANK_TOP_M = 150`
Category bonus	`2.0` when `ENABLE_CATEGORY_BOOST = True`
Reported classifier summary	train / validation / hold-out test accuracy table; production rows have no accuracy target

Table 2: Evaluation and reranking settings in the current saved notebook.

The notebook prints a split summary table together with classifier, validation, and hold-out test summary tables at runtime. Because those values are computed dynamically rather than hard-coded in the source file, this report documents the protocol and settings rather than freezing one transient metric snapshot in the text.

5 Conclusion

Our system evolved from basic lexical matching to a two-stage semantic retrieval pipeline centered on dense embeddings, aggressive caching, and cross-encoder reranking. In the current saved notebook snapshot, the active configuration is more disciplined than before: embedding-only first-stage retrieval at `top_k=7,500`, a stratified 60/30/10 query split, a category classifier fit only on the split-train queries, a cross-encoder trained only on the split-train queries, a soft category bonus of 2.0 when enabled, and consistent reranking at top 150 for validation, hold-out test, and production submission. This report now matches that notebook behavior directly, including the important fact that fitted query preprocessing is no longer learned on evaluation queries.